

Algorithm Module — Quiz Study Guide

Who this is for: You. P1 on the MDP team. Algorithm, Path Planning, Simulator.

Goal of this guide: Explain everything so clearly that you could teach it to someone who has never written code.

Table of Contents

1. [What Are We Actually Building?](#)
2. [The Playing Field](#)
3. [How We Describe "Where the Robot Is"](#)
4. [How the Robot Moves](#)
5. [The Big Problem: Getting From A to B](#)
6. [Dubins Paths — the Six Recipes](#)
7. [Where to Stand to Photograph an Obstacle](#)
8. [What Order Should We Visit the Obstacles?](#)
9. [What If the Shortest Path Exits the Arena?](#)
10. [Translating a Path into Robot Commands](#)
11. [Putting It All Together](#)
12. [How the Simulator Animates This](#)
13. [The Tests — Why They Exist and What They Prove](#)
14. [Quick-Fire Quiz Prep](#)

1. What Are We Actually Building?

Imagine a tiny robot car placed in a big square room (200 cm × 200 cm). Around the room there are 5 small blocks — called **obstacles**. Each block has a picture stuck to one of its sides. The robot needs to:

1. Drive up to each block and face the correct side (the side with the picture).
2. Take a "photo" from close range (20 cm away from the block face).
3. Visit **all 5 blocks** before time runs out.

4. Do it in the **shortest total distance** possible.

Your job is to write the software that figures out the route and drives the robot — entirely in simulation first (on screen using Python), before anything touches the real hardware.

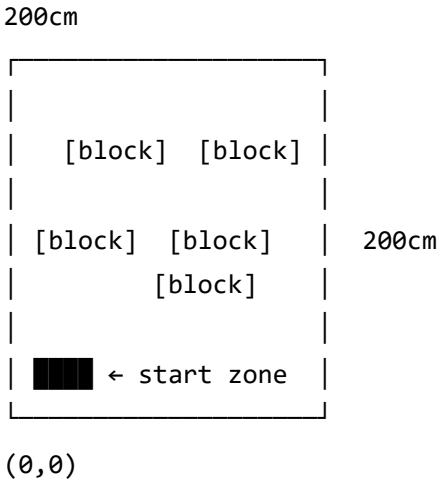
What you are graded on (B.1, B.2, B.3)

Grade item	Plain English
B.1	Show the arena, the blocks, and the robot moving on screen
B.2	The robot visits all 5 block pictures (in some order)
B.3	The robot visits them in the shortest possible order

2. The Playing Field

The arena is a grid

Think of the floor as a checkerboard. Each square on the checkerboard is **10 cm × 10 cm**. There are 20 squares across and 20 squares down, making the whole arena **200 cm × 200 cm**.



The robot always starts in the **bottom-left corner** (the "start zone"), specifically at position **(20 cm, 20 cm)** — the center of the 40×40 cm start zone.

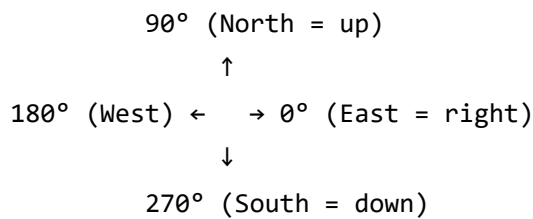
Measuring position

Every position in the arena is described as (**x cm from the left wall**, **y cm from the bottom wall**). So:

- **(0, 0)** = the very bottom-left corner
- **(200, 200)** = the very top-right corner
- **(100, 100)** = dead center of the arena

Measuring direction (angle)

The robot's direction is measured in **degrees**, going **counter-clockwise** (the opposite of how a clock ticks), starting from East (right):



So when we say the robot is facing **90°**, that means it's pointing straight up (North). The robot always **starts facing North (90°)**.

Why counter-clockwise? This is standard math convention (think of a unit circle from school). Our code uses this consistently everywhere.

The screen vs. the math (a gotcha)

In real life and in our math, $Y=0$ is at the **bottom** and increases upward (like a normal graph). On a computer screen, $Y=0$ is at the **top** and increases downward (like reading a page).

The function `cm_to_px()` in [simulator/arena.py](#) handles this flip automatically whenever we need to draw something:

```
def cm_to_px(x_cm, y_cm):  
    px = int(x_cm * CELL_PX / CELL_CM)  
    py = int(ARENA_PX - y_cm * CELL_PX / CELL_CM) # ← this line flips Y  
    return px, py
```

Example: The point (0, 0) in our math (bottom-left of the arena) becomes pixel (0, 800) on screen (also bottom-left, because the screen is 800px tall).

Tests that prove this:

```
assert cm_to_px(0, 0) == (0, 800) # bottom-left → stays bottom-left ✓
assert cm_to_px(0, 200) == (0, 0) # top-left math → top-left screen ✓
assert cm_to_px(200, 0) == (800, 800) # bottom-right → bottom-right ✓
assert cm_to_px(100, 100) == (400, 400) # center → center ✓
```

Key numbers to memorize (config.py)

Setting	Value	What it means
CELL_CM	10	Each grid square is 10 cm wide
GRID_SIZE	20	20×20 grid
TURN_RADIUS_CM	25	Minimum turning radius of the robot car
ROBOT_W_CM	20	Robot is 20 cm wide
ROBOT_H_CM	21	Robot is 21 cm tall
APPROACH_CM	20	Robot stops 20 cm from the obstacle face
START_X_CM	20	Robot starts 20 cm from left wall
START_Y_CM	20	Robot starts 20 cm from bottom wall
START_THETA	90	Robot starts facing North

3. How We Describe "Where the Robot Is"

We use four simple data containers defined in [simulator/types.py](#). Think of them as forms you fill in.

RobotState — the robot's ID card

```
@dataclass
class RobotState:
    x: float # how many cm from the left wall
    y: float # how many cm from the bottom wall
    theta: float # which direction it's facing (degrees, 0=East, 90=North)
```

Example: RobotState(x=20, y=20, theta=90) means:

"The robot is 20 cm from the left, 20 cm from the bottom, and facing North." That's the starting position.

Obstacle — the block's ID card

```
@dataclass
class Obstacle:
    x: int    # cm — where the LEFT edge of the block is
    y: int    # cm — where the BOTTOM edge of the block is
    face: str # which side has the picture: 'N', 'S', 'E', or 'W'
```

Example: Obstacle(x=50, y=50, face='N') means:

"There's a block whose bottom-left corner is at (50, 50). The picture is on its North (top) face."

The five blocks hardcoded in [simulator/planner.py](#) are:

```
Obstacle(x=50, y=50, face='N')    # block 1: picture faces up
Obstacle(x=100, y=60, face='E')   # block 2: picture faces right
Obstacle(x=150, y=80, face='S')   # block 3: picture faces down
Obstacle(x=80, y=130, face='W')   # block 4: picture faces left
Obstacle(x=130, y=130, face='N')  # block 5: picture faces up
```

Command — a single instruction to the robot

```
@dataclass
class Command:
    kind: str    # what to do (see table below)
    value: float # how much (cm for distance, degrees for rotation)
```

Command kind	Meaning	Value means
FW	Drive forward	how many cm
BW	Drive backward	how many cm
TL	Turn left on the spot (debug only)	how many degrees
TR	Turn right on the spot (debug only)	how many degrees
AL	Arc left — drive in a left curve	how many cm of arc

Command kind	Meaning	Value means
AR	Arc right — drive in a right curve	how many cm of arc

TL and TR are like spinning in place. The **real robot can't do this** (it's a car, not a tank). They exist for manual testing only. The planner only ever outputs FW, AL, and AR.

Example: `Command(kind='FW', value=40.0)` means *"drive straight forward 40 cm."*

DubinsPath — a description of a three-part journey

```
@dataclass
class DubinsPath:
    path_type: str    # which of the 6 recipes (e.g. 'LSL', 'RSR', etc.)
    seg1: float       # length of first part in cm
    seg2: float       # length of second part in cm
    seg3: float       # length of third part in cm
    total: float      # total length (seg1 + seg2 + seg3)
```

Every route our robot takes between two points is broken into exactly **three parts** (some of them might be zero length). More on this in Section 5.

4. How the Robot Moves

The movement functions live in [simulator/robot.py](#).

Moving in a straight line: `move_forward`

```
def move_forward(state, cm):
    rad = math.radians(state.theta)
    return RobotState(
        x = state.x + cm * math.cos(rad),
        y = state.y + cm * math.sin(rad),
        theta = state.theta,    # direction doesn't change
    )
```

Plain English: "Move `cm` centimetres in whatever direction `theta` points."

The `cos` and `sin` break the direction into X (horizontal) and Y (vertical) parts. If you're facing straight North (90°), $\cos(90^\circ) = 0$ and $\sin(90^\circ) = 1$, so you move purely vertically — no sideways movement at all.

Test example:

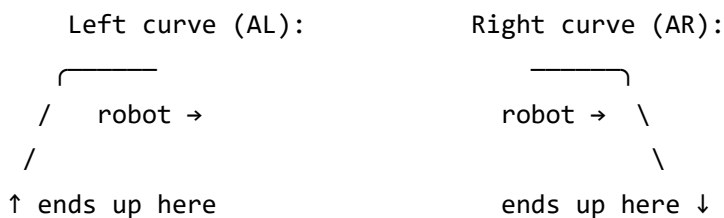
```
# Robot faces North (90°), moves 10 cm forward
state = RobotState(x=0, y=0, theta=90)
result = move_forward(state, 10.0)
# result.x ≈ 0    (didn't move sideways)
# result.y ≈ 10   (moved 10 cm upward = North) ✓
```

Moving **backward** is the same function with a negative distance:

```
result = move_forward(state, -10.0) # drives backward
```

Driving in a curve: `arc_step`

This is the most important movement function. The robot doesn't just go straight — it can sweep in a curve, like a car turning a corner.



```
def arc_step(state, ds, clockwise, r):
    sign = -1 if clockwise else 1
    theta_rad = math.radians(state.theta)
    new_theta_rad = theta_rad + sign * ds / r
    new_x = state.x + sign * r * (math.sin(new_theta_rad) - math.sin(theta_rad))
    new_y = state.y - sign * r * (math.cos(new_theta_rad) - math.cos(theta_rad))
    return RobotState(x=new_x, y=new_y, theta=math.degrees(new_theta_rad) % 360)
```

Plain English: The robot is driving along the edge of an invisible circle. `r` is the radius of that circle (25 cm). `ds` is how many centimetres it travels along that circle's edge. The robot's direction automatically changes as it curves.

Test examples:

```
# Start facing East (0°), arc LEFT a quarter circle (r=25 cm)
# Quarter circle arc length =  $(\pi/2) \times 25 \approx 39.3$  cm
state = RobotState(0, 0, 0)
result = arc_step(state, math.pi/2 * 25, clockwise=False, r=25)
# result  $\approx$  (25, 25, 90°)
# → moved forward-and-left, now facing North ✓
```

Picture it: you're driving East. You curve left (counter-clockwise) for a quarter circle. You end up facing North, 25 cm to the right of where you started and 25 cm up. That's exactly what (25, 25, 90°) describes.

```
# Same start, arc RIGHT a quarter circle
result = arc_step(state, math.pi/2 * 25, clockwise=True, r=25)
# result  $\approx$  (25, -25, 270°)
# → moved forward-and-right, now facing South ✓
```

```
# Full circle (360°) brings you back to exactly where you started
state = RobotState(10, 20, 45)
result = arc_step(state, 2 * math.pi * 30, clockwise=False, r=30)
# result  $\approx$  (10, 20, 45°) – back at start ✓
```

How the simulator moves the robot a tiny bit at a time: step_command

The simulator can't just teleport the robot. It moves it **2 cm at a time**, frame by frame (60 frames per second), so you can see it moving smoothly on screen.

```
def step_command(state, cmd, remaining):
    if cmd.kind == 'FW':
        advance = min(2.0, remaining)    # move at most 2 cm this frame
        return move_forward(state, advance), remaining - advance
    if cmd.kind == 'AL':
        advance = min(2.0, remaining)
        return arc_step(state, advance, clockwise=False, r=25), remaining - advance
    # ... same for BW, AR, TL, TR
```


remaining tells us how much of the current command is left. Each frame, it shrinks by 2 cm (or 3° for rotation). When it hits zero, the command is finished and the next one starts.

5. The Big Problem: Getting From A to B

Why is this hard?

The robot is not a person. It cannot:

- Jump sideways
- Spin in place
- Teleport

It is like a **car**. When a car turns, it sweeps a curve. The tighter the curve, the bigger the circle it traces. Our robot's **minimum turning radius is 25 cm** — it physically cannot turn in a tighter arc than that.

So the question is: *"What is the shortest possible route from pose A (position + direction) to pose B (position + direction), given that you can only go in curves of radius ≥ 25 cm or straight lines?"*

A mathematician named **Lev Dubins** solved this problem in 1957. His answer:

The shortest path is always made of at most 3 pieces. Each piece is either a straight line or a circular arc (with the minimum turning radius).

These are called **Dubins paths**.

The notation: L, R, S

- **L** = curve **Left** (counter-clockwise arc, radius = 25 cm)
- **R** = curve **Right** (clockwise arc, radius = 25 cm)
- **S** = go **Straight**

Every possible shortest path is some combination of these three letters, taken three at a time.

The six possible combinations

There are exactly **6 path types**. Think of them like 6 cooking recipes — given the same start and end, each recipe gives a different route, and you pick the shortest one.

Type	Pieces	What it looks like
LSL	Left curve → Straight → Left curve	Curve left, go straight, curve left again
RSR	Right curve → Straight → Right curve	Curve right, go straight, curve right again
LSR	Left curve → Straight → Right curve	Curve left, go straight, curve right
RSL	Right curve → Straight → Left curve	Curve right, go straight, curve left
RLR	Right curve → Left curve → Right curve	Three curves, no straight
LRL	Left curve → Right curve → Left curve	Three curves, no straight

The analogy: recipe book

Imagine you're driving from your house to a shop. You could:

- Go around the block left, then take the highway, then go around another block left → **LSL**
- Go around left, take the highway, go around right → **LSR**
- Wiggle through three roundabouts → **RLR**

You'd calculate the distance for each route and pick the shortest. That's exactly what `dubins_optimal()` does.

dubins_optimal in [simulator/dubins.py](#)

```
def dubins_optimal(q1, q2, r):
    candidates = [
        dubins_lsl(q1, q2, r),    # try recipe 1
        dubins_rsr(q1, q2, r),    # try recipe 2
        dubins_lsr(q1, q2, r),    # try recipe 3
        dubins_rsl(q1, q2, r),    # try recipe 4
        dubins_rlr(q1, q2, r),    # try recipe 5
        dubins_lrl(q1, q2, r),    # try recipe 6
    ]
    # pick the shortest one that actually worked (not None)
    return min((c for c in candidates if c is not None), key=lambda p: p.total)
```

Test — when the target is directly ahead, the answer is a straight line:

```

q1 = RobotState(0, 0, 0)          # at origin, facing East
q2 = RobotState(100, 0, 0)        # 100 cm ahead, also facing East
path = dubins_optimal(q1, q2, r=25)
assert abs(path.total - 100) < 0.1 # pure straight line, exactly 100 cm ✓

```

Test — the optimal is always $\leq$ every other candidate:

```

q1 = RobotState(0, 0, 0)
q2 = RobotState(50, 50, 90)
path = dubins_optimal(q1, q2, r=25)
# For every other formula: path.total ≤ candidate.total ✓

```

6. Dubins Paths — the Six Recipes

Each of the six functions takes two robot poses and a turning radius, and returns a `DubinsPath`. Here's how to understand each one without the heavy math.

Why some recipes can return `None`

Some path types are **geometrically impossible** in certain situations. Imagine trying to do an LSR path (curve left, straight, curve right) when the two positions are only 10 cm apart — the left-curving and right-curving circles would overlap, and there's no straight segment that can connect them. In those cases, the function returns `None`, meaning "this recipe doesn't work here." `dubins_optimal` simply skips those.

LSL — Left curve → Straight → Left curve

When would you use this? When both the start and end positions require you to exit / enter with a left turn. Like turning into a road from the left and turning left out of it.

```
def dubins_lsl(q1, q2, r):
    # Normalize distance by r (so radius=1 in the math)
    dx = (q2.x - q1.x) / r
    dy = (q2.y - q1.y) / r
    alpha = math.radians(q1.theta)
    beta = math.radians(q2.theta)
    tmp0 = dx + math.sin(alpha) - math.sin(beta)
    tmp1 = dy - math.cos(alpha) + math.cos(beta)
    p = math.sqrt(tmp0**2 + tmp1**2) # straight segment length (normalized)
    theta = math.atan2(tmp1, tmp0)
    t = _mod2pi(theta - alpha) # first arc angle
    q = _mod2pi(beta - theta) # third arc angle
    return DubinsPath('LSL', t*r, p*r, q*r, (t+p+q)*r)
```

What `_mod2pi` does: It keeps an angle in the range $[0, 2\pi]$. Think of it like a clock — going past 360° wraps back to 0° . Same idea, but in radians.

LSL can always return a result — there's always *some* way to do left-straight-left between any two poses.

Test — same start and end = zero path:

```
q = RobotState(0, 0, 0)
path = dubins_lsl(q, q, r=25)
assert path.total < 0.01 # no distance to cover → path length ≈ 0 ✓
```

RSR — Right curve → Straight → Right curve

The mirror image of LSL. Turns right both at the start and end. All the math signs flip.

```
tmp0 = dx - math.sin(alpha) + math.sin(beta) # signs flipped vs LSL
tmp1 = dy + math.cos(alpha) - math.cos(beta)
t = _mod2pi(alpha - theta)
q = _mod2pi(theta - beta)
```

Also always returns a result. Never returns `None`.

LSR — Left curve → Straight → Right curve

When would you use this? When you curve out from the start in one direction, go straight, and curve into the end from the other direction. Like an S-bend on a highway.

The key difference: Because you're curving in *opposite* directions, the two circles can overlap if the start and end are too close together. The code detects this:

```
p_sq = tmp0**2 + tmp1**2 - 4    # the "- 4" checks if circles overlap
if p_sq < 0:
    return None                # circles overlap — this path type is impossible here
```

Test — LSR on a U-turn:

```
q1 = RobotState(0, 0, 0)      # facing East
q2 = RobotState(80, 0, 180)    # 80 cm ahead, but now facing West (a U-turn)
path = dubins_lsr(q1, q2, r=25)
# Simulate the path step by step...
# Robot ends up at exactly (80, 0) facing West ✓
```

This U-turn is a perfect case for LSR: curve left away from start, go straight, curve right into the end pose.

RSL — Right curve → Straight → Left curve

Mirror of LSR. Same `p_sq < 0` early exit for when circles overlap.

Test — RSL on a U-turn in the other direction:

```
q1 = RobotState(0, 0, 180)    # facing West
q2 = RobotState(80, 0, 0)     # 80 cm away, facing East
path = dubins_rsl(q1, q2, r=25)
# Simulated path reaches (80, 0) facing East ✓
```

RLR — Right curve → Left curve → Right curve

When would you use this? Only when the two poses are very **close together** and doing three tight arcs is actually shorter than any option involving a straight line.

Think of it like a tight slalom: right, then immediately left, then right again. This is only possible when the two "outer" circles are within reach of each other (center-to-center distance $\leq 4r$).

```
d_sq = dx_c**2 + dy_c**2
if d_sq > 16.0:    # normalized by r, so "16" = (4r)2 / r2 = 16
    return None   # circles too far apart – RLR is impossible
```

Test — RLR on a tight reverse:

```
q1 = RobotState(0, 0, 315)    # facing roughly South-East
q2 = RobotState(30, 0, 135)   # 30 cm away, facing roughly North-West
path = dubins_rlr(q1, q2, r=25)
# Simulated path: three arcs, robot reaches q2 within 0.5 cm ✓
```

LRL — Left curve → Right curve → Left curve

Mirror of RLR. Same $4r$ distance check.

Test — LRL on a tight manoeuvre:

```
q1 = RobotState(0, 0, 45)
q2 = RobotState(30, 0, 225)
path = dubins_lrl(q1, q2, r=25)
# Simulated path reaches q2 within 0.5 cm ✓
```

Properties every Dubins path must satisfy

These are tested every time:

```
# The three segment lengths always add up to the total
assert abs(path.total - (path.seg1 + path.seg2 + path.seg3)) < 0.001

# No segment can have a negative length
assert path.seg1 >= 0
assert path.seg2 >= 0
assert path.seg3 >= 0
```

7. Where to Stand to Photograph an Obstacle

The problem

Each obstacle is a 10×10 cm block somewhere in the arena. One of its four sides (N, S, E, or W) has a picture. The robot's camera is at its front, and the robot photographs best from **20 cm away from the face**.

So for each obstacle, we need to compute: *"Where exactly should the robot stand, and which direction should it face, to get a good photo?"*

This computed position is called an **approach pose**.

The formula in [simulator/planner.py](#)

```
def obstacle_approach_pose(obs):
    cx = obs.x + CELL_CM / 2    # center x of the block = left edge + 5 cm
    cy = obs.y + CELL_CM / 2    # center y of the block = bottom edge + 5 cm
    d = CELL_CM / 2 + APPROACH_CM # 5 + 20 = 25 cm from center

    if obs.face == 'N':    # picture is on top
        # → robot stands ABOVE the block, faces DOWN toward it
        return RobotState(x=cx, y=cy + d, theta=270)

    if obs.face == 'S':    # picture is on bottom
        # → robot stands BELOW the block, faces UP toward it
        return RobotState(x=cx, y=cy - d, theta=90)

    if obs.face == 'E':    # picture is on right side
        # → robot stands to the RIGHT, faces LEFT toward it
        return RobotState(x=cx + d, y=cy, theta=180)

    # face == 'W':          picture is on left side
    # → robot stands to the LEFT, faces RIGHT toward it
    return RobotState(x=cx - d, y=cy, theta=0)
```

The simple rule:

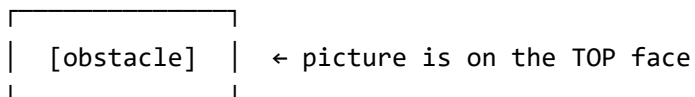
- Stand on the **same side as the face** (if face is North, stand North of the block)
- Face **toward** the block (face North → robot points South = 270°)

Diagram

face='N' case:

[ROBOT, facing 270° = South]

↓ ← robot points this way



d = 25 cm from center of block to center of robot

Worked examples from the tests

Obstacle(x=50, y=50, face='N'):

- Block center: $cx = 50 + 5 = 55$, $cy = 50 + 5 = 55$
- Distance: $d = 5 + 20 = 25$ cm
- Face = North $\rightarrow$ robot goes above: $y = 55 + 25 = 80$
- Robot faces South: $\theta = 270$
- **Result: RobotState(x=55, y=80, theta=270) ✓**

Obstacle(x=50, y=50, face='E'):

- Block center: $cx=55$, $cy=55$
- Face = East $\rightarrow$ robot goes to the right: $x = 55 + 25 = 80$
- Robot faces West: $\theta = 180$
- **Result: RobotState(x=80, y=55, theta=180) ✓**

The pattern (memorize this table)

Picture face	Robot stands...	Robot faces...	Robot theta
North (top)	Above the block	South (down)	270°
South (bottom)	Below the block	North (up)	90°
East (right)	Right of the block	West (left)	180°
West (left)	Left of the block	East (right)	0°

8. What Order Should We Visit the Obstacles?

The problem

We have 5 approach poses to visit. We can visit them in any order we like — but the order affects the total distance traveled. We want the shortest total route.

Example: Imagine 3 destinations: A is next door, B is across town, C is next door in the other direction.

- Route $A \rightarrow B \rightarrow C$: short, long, short = not bad

- Route B→A→C: long, short, short = wastes a trip

The order matters.

Why not use a clever algorithm?

With **5 obstacles**, there are **$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$ possible orderings**. A computer can calculate all 120 in a tiny fraction of a second. So we just try them all and pick the best. This is called **brute-force** — simple but correct.

Note: Famous routing problems like Google Maps use fancy algorithms because they have thousands of stops. We have 5. Brute force is perfectly fine.

The implementation in [simulator/planner.py](#)

```
def _hamiltonian_optimal_order(start, poses, r):
    best = []
    best_len = float('inf')    # start with "infinitely long" as our worst case

    for perm in itertools.permutations(poses):    # loops through all 120 orderings
        length = _total_dubins_length(start, list(perm), r)
        if length < best_len:
            best_len = length
            best = list(perm)    # this ordering is the best so far

    return best
```

`_total_dubins_length` adds up the Dubins path length from start → pose 1 → pose 2 → ... → pose 5 for a given ordering. We pick whichever ordering gives the smallest total.

Why is it called a "Hamiltonian path"? A Hamiltonian path is a route that visits every point exactly once. That's exactly what we need — visit all 5 blocks, each exactly once, in the shortest total distance.

Test examples

Does it visit the close one before the far one?

```

start = RobotState(0, 0, 0)
close = RobotState(10, 0, 0) # 10 cm away
far   = RobotState(100, 0, 0) # 100 cm away

result = _hamiltonian_optimal_order(start, [far, close], r=25)
assert result[0].x == close.x # close target is visited first ✓

```

Does it visit all poses and not miss any?

```

start = RobotState(0, 0, 90)
poses = [RobotState(50, 0, 0), RobotState(100, 0, 0), RobotState(150, 0, 0)]
result = _hamiltonian_optimal_order(start, poses, r=25)
assert len(result) == 3 # all 3 returned ✓
assert {(p.x, p.y) for p in result} == {(p.x, p.y) for p in poses} # same set ✓

```

Does it return all 5 for the real obstacles?

```

poses = [obstacle_approach_pose(obs) for obs in OBSTACLES]
result = _hamiltonian_optimal_order(start, poses, r=25)
assert len(result) == 5 # all 5 returned ✓

```

9. What If the Shortest Path Exits the Arena?

The problem

The shortest Dubins path between two poses is calculated purely by math — it doesn't know about the arena walls. Sometimes the mathematically shortest route would swing outside the 200×200 cm box.

We can't just ignore this — a robot that drives into the wall is a failed run.

The solution: try paths in order until one fits

The function `_dubins_bounded()` in [simulator/planner.py](#) works like this:

1. Compute all 6 path types and sort them from shortest to longest.
2. For each candidate, simulate it step-by-step and check if every point stays inside the arena.
3. Return the first one that fits (which is also the shortest that fits).

4. If none fit (edge case), return the shortest one anyway as a fallback.

```
def _dubins_bounded(q1, q2, r):
    candidates = sorted([... all 6 path types ...], key=lambda p: p.total)

    for path in candidates:
        if _path_in_bounds(q1, dubins_to_commands(path), r):
            return path          # shortest path that stays inside ✓

    return candidates[0]        # all paths exit – use shortest as fallback
```

How the boundary check works: `_path_in_bounds`

```
def _path_in_bounds(q1, cmds, r):
    x, y, theta = q1.x, q1.y, q1.theta
    step = 2.0      # check position every 2 cm along the path

    for cmd in cmds:
        remaining = cmd.value
        while remaining > 0.001:
            advance = min(step, remaining)
            # move x, y, theta by a small step
            ...
            remaining -= advance
            if not (0 <= x <= 200 and 0 <= y <= 200):
                return False    # oops, stepped outside the arena!

    return True    # stayed inside the whole time ✓
```

Plain English: Walk the path in 2 cm chunks. At each step, check: "Are we still inside the 200×200 box?" If any step exits the box, fail immediately.

10. Translating a Path into Robot Commands

Once we have a `DubinsPath` (e.g., `type=LSR, seg1=30, seg2=50, seg3=20`), we need to turn it into a list of `Command` objects that the simulator can actually execute.

The lookup table in `planner.py`

```
_SEGMENT_KINDS = {
    'LSL': ('AL', 'FW', 'AL'), # L=arc-left, S=forward, L=arc-left
    'LSR': ('AL', 'FW', 'AR'), # L=arc-left, S=forward, R=arc-right
    'RSL': ('AR', 'FW', 'AL'), # R=arc-right, S=forward, L=arc-left
    'RSR': ('AR', 'FW', 'AR'), # R=arc-right, S=forward, R=arc-right
    'LRL': ('AL', 'AR', 'AL'), # L=arc-left, R=arc-right, L=arc-left
    'RLR': ('AR', 'AL', 'AR'), # R=arc-right, L=arc-left, R=arc-right
}
```

Each path type maps to three command types. The function just looks up the type and pairs each command with its segment length:

```
def dubins_to_commands(path):
    k1, k2, k3 = _SEGMENT_KINDS[path.path_type]
    cmds = []
    for kind, seg in zip((k1, k2, k3), (path.seg1, path.seg2, path.seg3)):
        if seg > 0.01: # skip zero-length segments
            cmds.append(Command(kind, seg))
    return cmds
```

Test example — LSL path:

```
path = DubinsPath(path_type='LSL', seg1=30, seg2=50, seg3=20, total=100)
cmds = dubins_to_commands(path)
# Result: [Command('AL', 30), Command('FW', 50), Command('AL', 20)]
# → arc left 30 cm, drive straight 50 cm, arc left 20 cm
```

Test example — zero-length segment is skipped:

```
path = DubinsPath(path_type='LSL', seg1=30, seg2=0.0, seg3=20, total=50)
cmds = dubins_to_commands(path)
# Result: [Command('AL', 30), Command('AL', 20)]
# → the straight segment was 0 cm so it was dropped
```

11. Putting It All Together

The function `get_commands()` in [simulator/planner.py](#) is the **master function** that ties every other piece together. When you run the simulator, this is called once to produce the entire movement script.

```
def get_commands(obstacles):
    # Step 1: Start position
    start = RobotState(x=20, y=20, theta=90)

    # Step 2: For each obstacle, figure out WHERE to stand to photograph it
    poses = [obstacle_approach_pose(obs) for obs in obstacles]

    # Step 3: Find the shortest ORDER to visit all 5 poses (tries all 120 orderings)
    ordered = _hamiltonian_optimal_order(start, poses, TURN_RADIUS_CM)

    # Step 4: For each leg of the journey, find the shortest in-bounds Dubins path
    current = start
    cmds = []
    for pose in ordered:
        path = _dubins_bounded(current, pose, TURN_RADIUS_CM)
        cmds += dubins_to_commands(path)  # translate path → commands
        current = pose

    return cmds
```

Step-by-step in plain English

1. **Start** the robot at (20, 20) facing North.
2. **Compute the 5 target poses** — one for each obstacle (where to stand and which way to face for the photo).
3. **Try all 120 orderings** of those 5 poses. Calculate the total Dubins distance for each ordering. Keep the shortest.
4. **Build the command list**: for each consecutive pair of poses in the chosen order, find the shortest route that stays inside the arena, then translate that route into FW/AL/AR commands.
5. **Return the full command list** — the simulator will play this back frame by frame.

The big end-to-end test

```
def test_get_commands_reaches_final_approach_pose():
    start = RobotState(x=20, y=20, theta=90)
    cmds = get_commands(OBSTACLES)

    # Actually simulate every single command
    state = start
    for cmd in cmds:
        remaining = cmd.value
        while remaining > 0.001:
            state, remaining = step_command(state, cmd, remaining)

    # Check: does the robot end up near an approach pose?
    poses = [obstacle_approach_pose(obs) for obs in OBSTACLES]
    closest = min(poses, key=lambda p: math.hypot(state.x - p.x, state.y - p.y))
    assert math.hypot(state.x - closest.x, state.y - closest.y) < 2.0 # within 2 cm ✓
```

What this test proves: The entire chain — approach pose calculation → Hamiltonian ordering → Dubins path computation → command conversion → step-by-step simulation — is accurate enough that the robot lands within 2 cm of its intended destination.

12. How the Simulator Animates This

The file [simulator/main.py](#) runs the visual simulator. Here's the core loop explained:

```

commands = get_commands(OBSTACLES)    # compute the full journey once at startup
queue    = list(commands)              # copy into a queue we'll consume
active   = None                        # the currently executing command
remaining = 0.0                        # how much of it is left

while running:    # runs 60 times per second (60 FPS)

    # Handle keyboard input
    # Space = pause/unpause, R = reset to start, Q/Esc = quit

    if not paused:
        if active is None and queue:
            active = queue.pop(0)        # grab the next command from the list
            remaining = active.value     # reset the counter

        if active is not None:
            state, remaining = step_command(state, active, remaining)
            if remaining <= 0:
                active = None           # command is done, move to next one

    # Draw everything
    draw_arena(screen)
    draw_obstacles(screen, OBSTACLES)
    draw_robot(screen, state)
    pygame.display.flip()              # show the new frame
    clock.tick(60)                     # wait until the next 1/60 second

```

Plain English: Every 1/60th of a second, move the robot 2 cm (or 3°) along the current command, redraw the screen, and check if the current command is done. When done, pick the next one. Repeat until all commands are finished.

Controls:

Key	What happens
Space	Pause or resume the animation
R	Reset robot to start position
Q or Esc	Close the simulator

13. The Tests — Why They Exist and What They Prove

The tests in [simulator/tests/test_logic.py](#) are your safety net. Each test proves one small thing works correctly. Together they prove the whole system is correct.

Coordinate system tests — "does the screen match the math?"

```
assert cm_to_px(0, 0) == (0, 800) # bottom-left stays bottom-left ✓
assert cm_to_px(0, 200) == (0, 0) # top-left (math) → top-left (screen) ✓
assert cm_to_px(200, 0) == (800, 800) # bottom-right → bottom-right ✓
assert cm_to_px(100, 100) == (400, 400) # center → center ✓
```

Movement tests — "does the robot go where it should?"

```
# Face North, drive 10 cm → should move straight up (y increases, x unchanged)
result = move_forward(RobotState(0, 0, 90), 10)
assert abs(result.x) < 0.001 # x unchanged ✓
assert abs(result.y - 10) < 0.001 # y went up by 10 cm ✓

# Arc left quarter circle from (0,0) facing East → end up at (25, 25) facing North
result = arc_step(RobotState(0, 0, 0), math.pi/2 * 25, clockwise=False, r=25)
assert abs(result.x - 25) < 0.01
assert abs(result.y - 25) < 0.01
assert abs(result.theta - 90) < 0.01 # now facing North ✓

# Full circle returns to start
result = arc_step(RobotState(10, 20, 45), 2*math.pi*30, clockwise=False, r=30)
assert abs(result.x - 10) < 0.1 # back where we started ✓
assert abs(result.y - 20) < 0.1
```

Dubins tests — "does the path math produce correct paths?"

```
# Straight target = straight path (no arcs needed)
path = dubins_optimal(RobotState(0,0,0), RobotState(100,0,0), r=25)
assert abs(path.total - 100) < 0.1 # 100 cm, no curving ✓

# Segments must sum to total
assert abs(path.total - (path.seg1 + path.seg2 + path.seg3)) < 0.001 ✓

# No segment is negative
assert path.seg1 >= 0 and path.seg2 >= 0 and path.seg3 >= 0 ✓

# Simulate the path and reach the destination
# (tests for LSR, RSL, RLR, LRL each simulate the path and check endpoint)
assert abs(state.x - q2.x) < 0.5 # within half a centimetre ✓
assert abs(state.y - q2.y) < 0.5 ✓
```

Approach pose tests — "does the robot line up with the obstacle correctly?"

```
# face='N': robot should be directly above, facing down (270°)
obs = Obstacle(x=50, y=50, face='N')
pose = obstacle_approach_pose(obs)
assert abs(pose.x - 55) < 0.01 # aligned with block center ✓
assert abs(pose.y - 80) < 0.01 # 25 cm above center ✓
assert abs(pose.theta - 270) < 0.01 # facing South (down toward the block) ✓
```

Hamiltonian ordering tests — "does the route optimizer work?"

```
# Given a close and far target, close should be visited first
result = _hamiltonian_optimal_order(start, [far, close], r=25)
assert result[0].x == close.x ✓

# All 5 real obstacle approach poses are returned
result = _hamiltonian_optimal_order(start, five_poses, r=25)
assert len(result) == 5 ✓
```

14. Quick-Fire Quiz Prep

The five concepts you must be able to explain out loud

1. What is a Dubins path and why do we need it?

The robot is like a car — it can't spin in place or go sideways. It must follow curves with a minimum turning radius of 25 cm. A Dubins path is the mathematically shortest route between two positions-with-directions (called poses) under this constraint. Every such shortest path is always made of exactly three pieces: each piece is either a left curve, a right curve, or a straight line.

2. What are the 6 Dubins types?

LSL, RSR, LSR, RSL (these have a straight middle segment) and LRL, RLR (these have three arcs in a row). L = left arc, R = right arc, S = straight.

3. When does a Dubins function return `None` ?

When the path type is geometrically impossible for those two poses. LSR and RSL return `None` when the start and end are too close together (the two arcs would overlap, leaving no room for a straight segment). RLR and LRL return `None` when the poses are too far apart for three arcs to connect.

4. How does the route ordering work?

There are 5 obstacles to visit, giving $5! = 120$ possible orderings. We try all 120, calculate the total Dubins path length for each, and keep the shortest. This is called brute-force — simple and correct because 120 is such a small number.

5. What is an approach pose and how is it calculated?

It's the exact (x, y, direction) the robot needs to be in to photograph a given obstacle face. It's calculated by finding the center of the obstacle, stepping 25 cm out in the direction of the face, and pointing the robot back toward the face. For example, a North-facing picture → robot stands 25 cm above it, facing South (270°).

The two functions that differ (and why)

Function	What it does
<code>dubins_optimal()</code>	Finds the globally shortest Dubins path, ignoring walls

Function	What it does
<code>_dubins_bounded()</code>	Finds the shortest Dubins path that stays inside the arena

`_dubins_bounded` is what the actual planner uses. It tries paths in order from shortest to longest and picks the first one that doesn't exit the 200×200 arena.

Numbers to know by heart

What	Value
Arena size	200 × 200 cm
Grid	20 × 20 cells, each 10 × 10 cm
Robot start	(20 cm, 20 cm), facing 90° (North)
Turning radius	25 cm
Photo distance	20 cm from face surface (= 25 cm from block center)
Robot size	20 × 21 cm
Orderings checked	5! = 120
Steps per frame	2 cm (distance) or 3° (rotation) at 60 FPS

Angle cheat sheet

Direction	Degrees
East (right)	0°
North (up)	90°
West (left)	180°
South (down)	270°

Angles go **counter-clockwise**. Clockwise is negative (or equivalently: 360° – the clockwise angle).

File-by-file: what does each file own?

File	One-sentence job
simulator/config.py	All the magic numbers in one place
simulator/types.py	The four data containers (RobotState, Obstacle, Command, DubinsPath)
simulator/dubins.py	The six Dubins path formulas + <code>dubins_optimal</code>
simulator/planner.py	Approach poses, visit ordering, bounded path selection, and <code>get_commands</code>
simulator/robot.py	Frame-by-frame movement: <code>move_forward</code> , <code>arc_step</code> , <code>step_command</code>
simulator/arena.py	Drawing the arena on screen + the cm-to-pixel coordinate flip
simulator/main.py	The main loop — plays the animation 60 times per second
simulator/tests/test_logic.py	All the tests that prove every piece works

Likely quiz questions and short answers

Q: Why can't the robot spin in place?

A: It uses Ackermann steering (like a car). The front wheels steer by turning, not by pivoting the whole body. So it must always move along a curve, never rotate on the spot.

Q: Why is brute-force okay for 5 obstacles?

A: $5! = 120$ is tiny. It runs in milliseconds. Fancy algorithms are only needed when you have thousands of stops (like GPS routing). We don't.

Q: What does `theta=270` mean?

A: The robot is facing South (downward on screen). Our angle system starts at 0° =East and goes counter-clockwise, so 270° is one full turn minus a quarter = South.

Q: What happens when `dubins_1sr` returns `None` ?

A: The `dubins_optimal` function filters it out and picks the shortest path among the ones that did work. `_dubins_bounded` does the same.

Q: What is `_mod2pi` doing?

A: Wrapping an angle back into the range $[0, 2\pi]$. Like how a clock resets to 12:00 after going past

midnight — angles wrap around after a full circle.

Q: The test checks the robot ends within 2 cm of its target. Why not exactly 0?

A: The simulator moves in 2 cm steps, so there's always a tiny rounding error. Anything under 2 cm is close enough to prove the math is correct.